

Guía de estudio: cómo funciona el Dashboard del TP N.º 3

Estadística y Probabilidades — Explicación del código, parte por parte, para poder responder con seguridad si te preguntan cómo lo hiciste.

¿Qué vas a encontrar acá?

Este documento resume, en orden de importancia, cómo funciona el código del dashboard: primero lo que seguro te van a preguntar (consumo de la API, arrays, Chart.js), y al final lo que es puramente estético (CSS) y no hace falta que expliques de memoria.

Sección	Importancia
1. Estructura general del archivo	Media
2. Consumo de la API (fetch / async-await)	Alta ★★★
3. Arrays y sus métodos (forEach, map, reduce)	Alta ★★★
4. Transformar datos antes de graficar	Alta ★★★
5. Chart.js: la receta de cada gráfico	Media-Alta ★★
6. Manejo de errores	Media ★★
7. CSS / diseño visual	Baja ★

1. Estructura general del archivo

Es un único archivo **.html** con tres partes, como cualquier página web:

- **<head>**: contiene el `<script>` con el código de Chart.js pegado adentro (para no depender de internet) y el `<style>` con el CSS.
- **<body>**: el HTML real — el header, las tarjetas de KPIs, y los `<canvas>` donde se dibujan los gráficos.
- **<script> final**: acá vive toda la lógica — pedir los datos y dibujar los gráficos.

```
<canvas id="chartAlumnadoNivel"></canvas>
```

Cada gráfico necesita un `<canvas>` en el HTML. Chart.js "dibuja" adentro de ese canvas. Por eso cada gráfico se busca en JS por su **id**.

2. Consumo de la API (lo más importante)

La función base que trae los datos de cada endpoint:

```
async function fetchData(endpoint){
  const res = await fetch(`${API_BASE}/${endpoint}/${ID}`);
  const json = await res.json();
  return json.data;
}
```

fetch(url)

Es la función nativa de JavaScript para pedir datos por HTTP (en este caso, un GET, tal como pide el enunciado). La URL se arma con template strings (los backticks), que concatenan texto:

apidemo.geoeducacion.com.ar/api/testing/estudiantes/1.

await y async

await significa "esperá a que esto termine antes de seguir". **fetch** es **asíncrono**: no sabemos cuánto va a tardar internet en responder, así que JavaScript no se queda frenado esperando — pero con **await** le decimos "en esta línea, esperá el resultado antes de continuar". Toda función que use **await** adentro tiene que estar marcada como **async**: es una regla del lenguaje.

res.json()

La respuesta del fetch llega primero como texto plano. **.json()** la convierte en un objeto de JavaScript que se puede usar (arrays, propiedades, etc). Se devuelve **json.data**, que es el array que en el enunciado venía adentro de "data": [...].

Promise.all: pedir varios endpoints a la vez

```
const [estudiantes, asistencia, historial, calificaciones, comunicados] =
  await Promise.all([
    fetchData('estudiantes'),
    fetchData('asistencia'),
    fetchData('historial_asistencia'),
    fetchData('calificaciones'),
    fetchData('comunicados')
  ]);
```

Hay que pedirle datos a **5 endpoints distintos**. Se podría hacer uno atrás del otro (esperar el primero, después el segundo...) pero sería más lento. **Promise.all** los dispara **todos a la vez** y espera a que terminen todos juntos. Con la desestructuración [estudiantes, asistencia, ...] cada resultado cae ordenado en su propia variable, en el mismo orden en que se pidieron.

■ Si te preguntan "¿por qué no hacer cinco fetch por separado, uno tras otro?" — la respuesta es rendimiento: con *Promise.all* las 5 peticiones viajan en paralelo, no en fila.

3. Arrays y sus métodos

Un **array** es una lista ordenada de elementos. Casi todos los datos que devuelve la API son arrays. Por ejemplo:

```
"data": [  
  { "id": 1, "nombre": "Martinez", ... },  
  { "id": 2, "nombre": "Gomez", ... },  
  { "id": 3, "nombre": "Lopez", ... }  
]
```

Eso [] es un array: una caja con varios elementos (acá, objetos con datos de cada estudiante) uno atrás de otro, en orden, cada uno con una posición (índice) que empieza en 0.

Método	Qué hace
.forEach()	Recorre cada elemento del array, sin devolver nada nuevo.
.map()	Recorre cada elemento y devuelve un array nuevo transformado.
.reduce()	Recorre y "acumula" todo en un solo valor (ej: una suma total).

Estos tres son la base para procesar cualquier lista de datos en JavaScript, no solo en este TP.

4. Transformar los datos antes de graficar

Los datos que da la API **no vienen listos para graficar**: hay que procesarlos primero. Ejemplo — "Composición del alumnado por nivel":

```
const nivelCounts = {};  
estudiantes.forEach(e => {  
  nivelCounts[e.nivel] = (nivelCounts[e.nivel] || 0) + 1;  
});
```

Se recorre el array de estudiantes uno por uno (forEach) contando cuántos hay de cada nivel (Inicial / Primario / Secundario), armando un objeto tipo {Primario: 12, Secundario: 8}. Chart.js necesita esa cuenta ya hecha, no se le puede pasar la lista cruda de alumnos.

Otro ejemplo, para el total de asistencia general se suman todos los cursos:

```
const totalPresentes = asistencia.reduce((a,c) => a + c.presentes, 0);
```

.reduce() acumula un valor recorriendo el array. Aquí suma el campo **presentes** de todos los cursos en un solo número total. Es, literalmente, una sumatoria — por eso es de los métodos que más preguntan en una materia de Estadística.

5. Chart.js: la receta de cada gráfico

Todos los gráficos siguen la misma fórmula. Una vez que se entiende uno, se entienden todos:

```
new Chart(document.getElementById('chartAsistenciaGeneral'), {  
  type: 'pie', // tipo de grafico  
  data: {  
    labels: ['Presentes', 'Ausentes'], // que dice cada porcion  
    datasets: [{  
      data: [totalPresentes, totalAusentes], // los numeros  
      backgroundColor: [...] // colores  
    }]  
  },  
  options: { ... } // detalles visuales (leyenda, etc)  
});
```

type puede ser pie, doughnut (dona), bar o line. Se elige según qué tipo de dato es:

Tipo	Se usa para...
pie / doughnut	Partes de un total (composición, porcentajes).
bar	Comparar categorías entre sí (curso A vs curso B).
line	Evolución en el tiempo (mes a mes).

labels y **data** tienen que tener el mismo orden: el label en la posición 0 corresponde al dato en la posición 0.

Caso especial: barras apiladas al 100%

Usado en "Comparativa de calificaciones por curso":

```
scales: { x: { stacked: true }, y: { stacked: true, max: 100 } }
```

stacked: true hace que las barras de "aprobados" y "desaprobados" se apilen una arriba de la otra en vez de ir lado a lado. Con **max: 100** se fuerza a que el eje llegue hasta 100%, porque son porcentajes.

6. Manejo de errores

```
try{  
  // todo el codigo de arriba  
}catch(err){  
  setStatus('No se pudieron cargar los datos: ' + err.message, true);  
}
```

Es buena práctica: si la API está caída o no hay internet, el **catch** agarra el error y muestra un mensaje en pantalla en vez de que la página quede rota sin explicación. Además se agregó un `window.addEventListener('error', ...)` como red de seguridad extra, por si el error ocurre *fuera* del try/catch (por ejemplo, si Chart.js no llega a cargar).

7. CSS — lo menos importante

Todo el bloque de estilos (colores, bordes redondeados, el grid que acomoda las tarjetas) es **estética pura**. No tiene lógica de estadística ni de programación "dura". Vale la pena mencionar solo dos cosas si preguntan:

- Se usa **CSS Grid** (`display:grid; grid-template-columns:repeat(3,1fr)`) para que las tarjetas se acomoden solas en 3 columnas.
- El diseño es **responsive**: con `@media` se reordena a 1 columna en el celular.

■ Si preguntan "¿por qué Chart.js y no otra librería?": el enunciado no exige una en particular, y Chart.js es de las más simples de usar — con un objeto de configuración (`type`, `data`, `options`) alcanza, sin aprender un lenguaje de gráficos nuevo.

Guía generada a partir de la explicación del código del Dashboard — TP N.º 3 de Estadística y Probabilidades, IDRA.